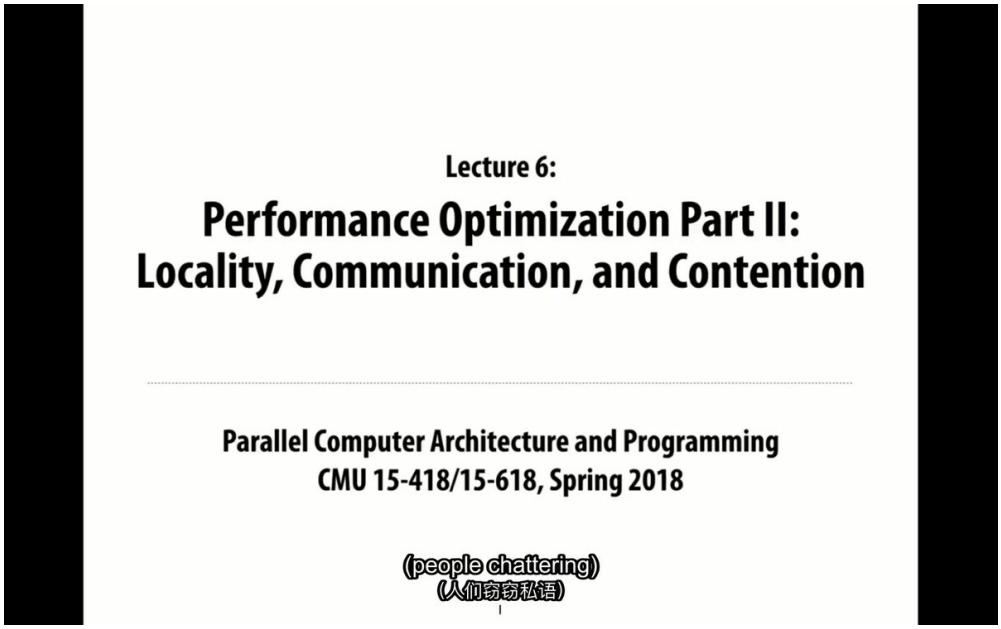
课程笔记

局部性、通信与竞争：数据搬运的真实代价

CMU 15-418/618 并行计算机架构与编程 · Lecture 8（Spring 2018）

lecture-to-notes

2026 年 10 月 28 日



视频作者/频道：CMU 15-418/618（B 站搬运：alexphil）
发布日期：2018 年春季学期
视频时长：1 小时 27 分 2 秒
视频链接：`https://www.bilibili.com/video/BV1vr4y1Z78J/`

目录

1 通信：并行程序里最容易被低估的成本 3
1.1 为什么通信能抹掉并行收益 . . . 3
1.2 一个把通信暴露得最清楚的例子 . . . 3
1.3 每块需要什么 . . . 4
1.4 收敛判断需要一次全局归约 . . . 5
1.5 本章小结 . . . 6

2 发送与接收的几种约定 6
2.1 阻塞发送：会合 . . . 6
2.2 死锁与奇偶解法 . . . 6
2.3 非阻塞发送 . . . 7
2.4 本章小结 . . . 7

3 延迟与带宽：两个必须分开的量 8
3.1 公路类比 . . . 8
3.2 两个量的定义 . . . 9
3.3 洗衣与流水线 . . . 10
3.4 处理器也是这么做的 . . . 11
3.5 本章小结 . . . 12

4 通信耗时的模型 12
4.1 固定开销加传输时间 . . . 12
4.2 有效带宽 . . . 14
4.3 最慢链路与流水线化通信 . . . 14
4.4 本章小结 . . . 15

5 把并行系统看成扩展的内存层次 15
5.1 缓存既是资源也是成本 . . . 15
5.2 固有通信与人为通信 . . . 16
5.3 算术强度：把通信与计算放在一起看 . . . 17
5.4 本章小结 . . . 18

6 分块形状如何决定算术强度 18
6.1 行分块 . . . 18
6.2 交叉分配：一个反面教材 . . . 18
6.3 方形分块：面积与周长之比 . . . 19
6.4 固有通信也可以减少 . . . 20
6.5 本章小结 . . . 20

7 人为通信：缓存行、缺失与遍历顺序 20
7.1 最小传输粒度 . . . 20
7.2 第四类缺失：通信缺失 . . . 21
7.3 从工作集看通信 . . . 21

7.4 行主序遍历的容量缺失 22
7.5 改遍历顺序与循环融合 22
7.6 数据并行写法与性能的张力 24
7.7 本章小结 24

8 竞争与伪共享 **25**
8.1 伪共享的机制 25
8.2 原子操作与热点 25
8.3 本章小结 25

9 总结与延伸 **25**

1　通信：并行程序里最容易被低估的成本

上一讲解决了如何把工作分下去。这一讲处理分配完成之后真正决定性能的部分：数据如何在处理器之间流动。

1.1　为什么通信能抹掉并行收益

讲者开门见山地描述了最常见的失败模式：你可能对如何把问题切成若干块有一个很好的想法，也确实并行起来了，然后被来回搬运数据的成本淹没，把并行带来的机会全部抵消掉。

这种成本出现在所有形态的并行系统上。共享内存的多核处理器里，它是缓存与总线上的流量；消息传递的大系统里，它是网络传输；而 GPU 只是这个问题的一种特殊口味——box 内部有些部分像共享内存，另一些则像消息传递，必须显式搬数据才能继续算。

三者的差别其实只在”距离”上。讲者说这些东西位于机器具体距离之上的更高层次，它们的共同问题是同一件事。

通信成本与并行收益是同一枚硬币的两面

切分带来并行，切分也带来跨块的数据需求。因此不存在”只提高并行度而不同时增加通信”的方案。判断一次并行化是否值得，必须同时算出两笔账：计算被分摊到多少份，以及块之间要搬多少数据。只算前一笔，就会得到上一讲那种 4 核比 2 核更慢的结果。

1.2　一个把通信暴露得最清楚的例子

讲者选消息传递版的网格求解器作为主线，理由很直接：在这个模型里通信是显式的，每一条数据交换都写在代码里，无法假装不存在。

问题本身是求每个网格点与其上下左右四个邻居的平均值。实现上加了一圈边缘行列，这样边界点也有合理的邻居值，不必为它们写特殊判断。因此数据是 $(N+2)\times(N+2)$ 的网格，实际计算的是内部的 $N\times N$ 区域。

加一圈边缘是为了消除边界分支

边界点缺少某个方向的邻居，若不特殊处理，内层循环里就要为每个元素判断”我是否在边界”。这会在最热的循环里引入分支，而且让向量化失效。加一圈哨兵行列把这些判断移出内层循环，代价只是少量存储与一次边界填充通信。

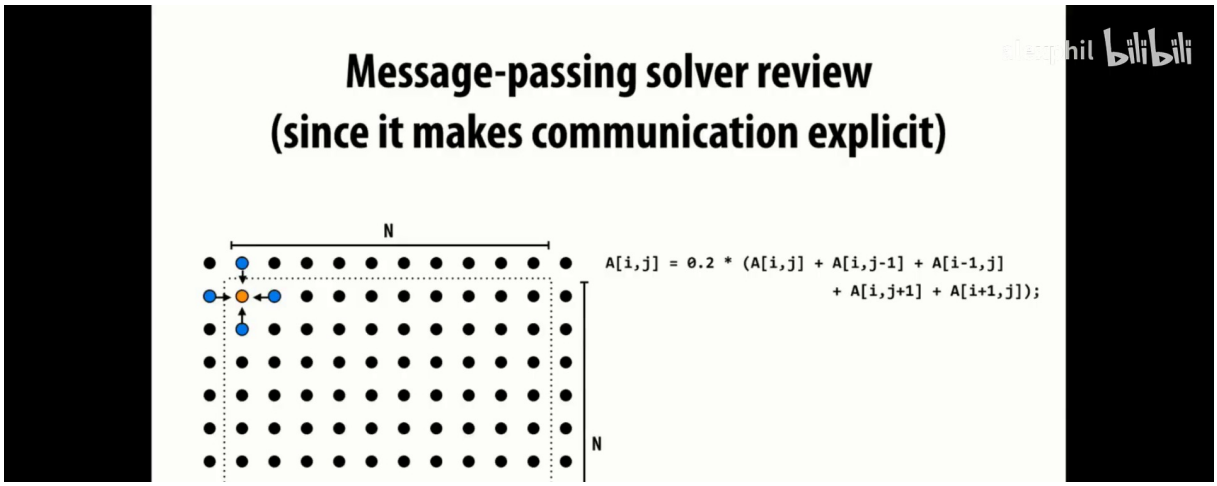


图 1: 消息传递求解器回顾：通信在这个模型里是显式的[1]

最自然的分法是按行切成块，块数等于可用线程数。为简化，讲者假设线程数能整除 N。

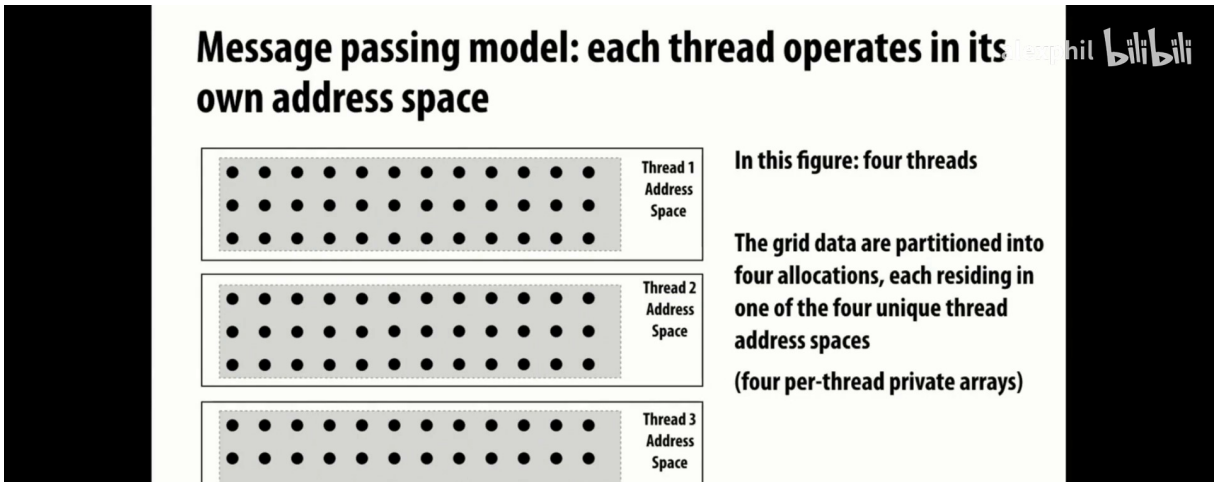


图 2: 消息传递模型：每个线程在自己的地址空间内运行[2]

1.3 每块需要什么

在消息传递世界里，每个块跑在一个独立处理器上，跨处理器取数据只能靠发消息。

假设是迭代计算：对全网格做一遍平均，更新状态，再重复，直到满足收敛条件。

那么第 2 号线程要算它那几行，就需要拿到它上方和下方相邻块的那一行数据。除了最上和最下的块，每个线程都要接收两组数据，也要向外发出两组——各一行。

[1] 视频画面时间区间：00:07:30–00:07:45。若为裁剪图，时间区间仍对应原视频画面。

[2] 视频画面时间区间：00:09:00–00:09:15。若为裁剪图，时间区间仍对应原视频画面。

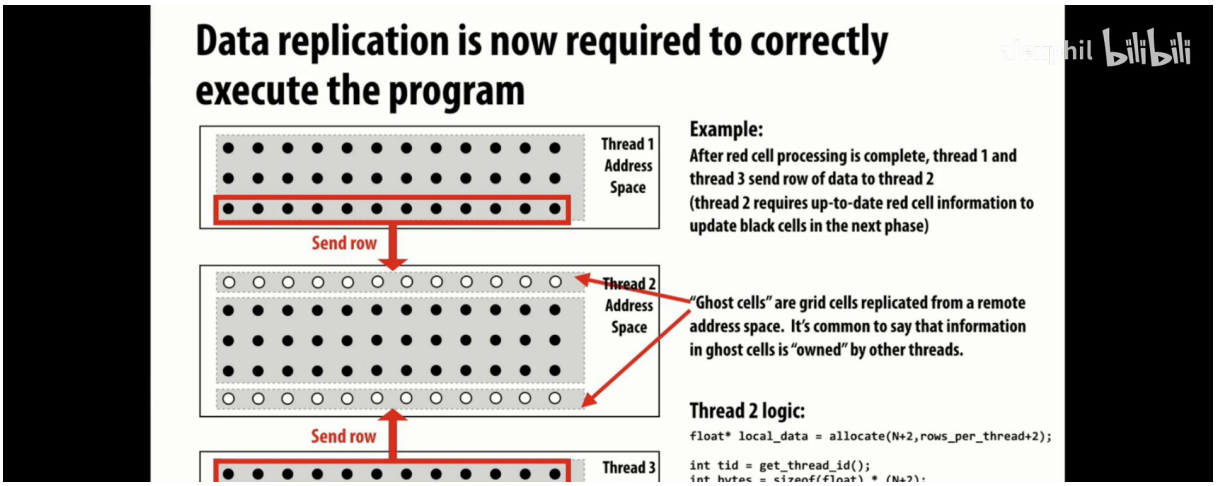


图 3: 消息传递中正确执行需要数据复制：每个线程保存自己的块与邻居的边界[3]

因为无法共享内存，每个线程必须自己保存一份邻居边界数据的副本。这就是”复制”的必要性。

1.4 收敛判断需要一次全局归约

除了取邻居数据，还有第二种通信。收敛条件是检查每个网格点的新旧差值、取绝对值、在全网格上求和，低于阈值就停止。

在消息传递里这必须分两步：所有线程把自己的差值发给 0 号线程；0 号线程负责接收并累加，判断是否足够小；然后把”继续”或”结束”的状态广播给所有线程。

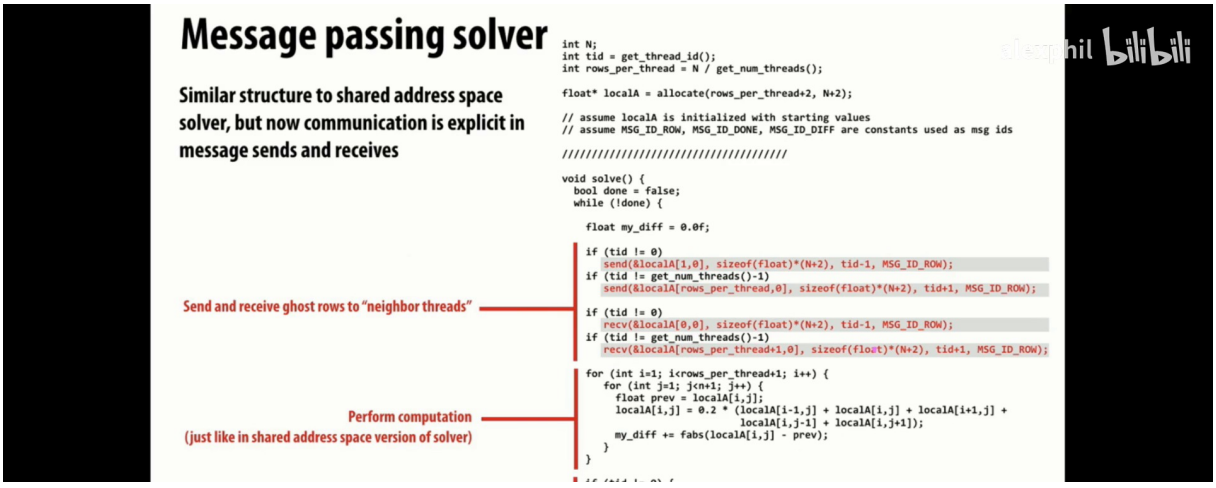


图 4: 消息传递求解器的代码：邻居交换与收敛归约两种通信并存[4]

于是这份程序里同时存在两种通信模式：邻接交换（局部、可并行）与主从归约广播（全局、串行化）。讲者说下一步看 MPI 时会发现，这类程序用 MPI 写起来相当容易。

[3]视频画面时间区间：00:10:00–00:10:15。若为裁剪图，时间区间仍对应原视频画面。

[4]视频画面时间区间：00:12:30–00:12:45。若为裁剪图，时间区间仍对应原视频画面。

1.5 本章小结

通信成本足以抹掉并行收益，而它在三种并行形态中本质相同，只是距离不同。网格求解器把这件事暴露得最清楚：按行分块后每个线程必须保存邻居边界数据的副本，收敛判断还需要一次把所有差值汇总到 0 号线程的全局归约。

既然通信无法避免，下一个问题是它的耗时由什么决定。这需要先把”延迟”与”带宽”这两个常被混用的量分清楚。

2 发送与接收的几种约定

在讨论性能之前，讲者先交代消息传递原语的语义差异，因为这直接决定了程序能否正确运行。

2.1 阻塞发送：会合

从系统实现角度看最直接的是阻塞发送，也叫会合：发送方与接收方必须同步，传输才能发生。

软件上的表现是双向的。接收时如果消息还没到，你会等待；而发送时，代码也会阻塞，直到接收方确认收到。这像一次握手：发送方想发、接收方想收，双方达成一致，数据被拷贝过去，然后各自返回。

讲者指出系统实现者喜欢这种方式，因为它要求最少的缓冲——不需要在接收方跑得比发送方慢时囤积数据。他还提到可以让发送方领先若干条消息的变体，但会复杂得多。

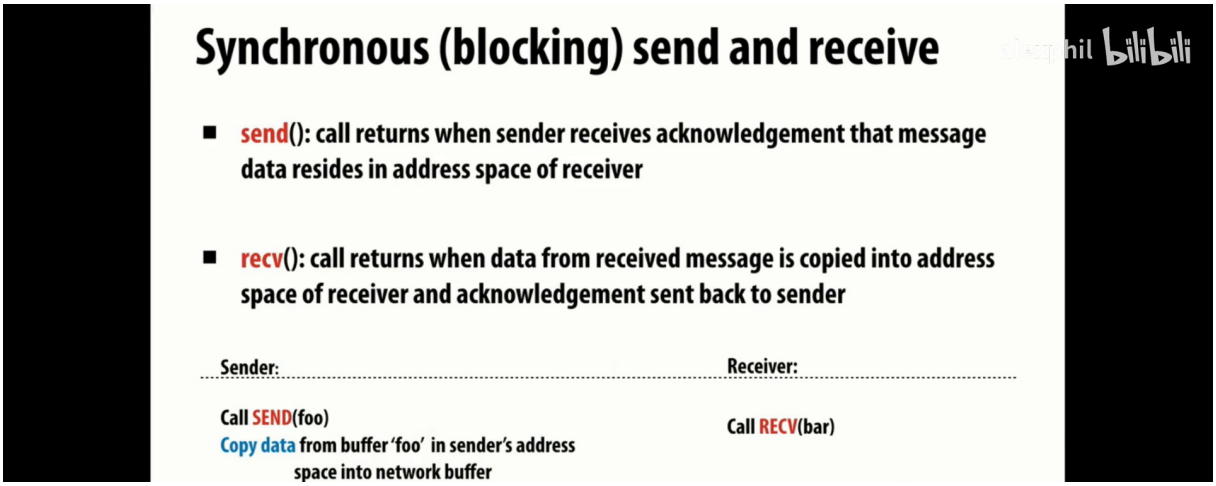


图 5: 同步（阻塞）发送与接收：调用在收到确认后返回[5]

2.2 死锁与奇偶解法

学生很快指出风险：如果所有人都在发、没人在收，就全部挂死。讲者确认这正是阻塞发送的典型陷阱。

而在网格求解器里，那段代码如果直接用阻塞发送，恰好会死锁——一开始除最上面的线程外，所有线程都在尝试发两条消息，而没有任何线程在尝试接收。

解法是奇偶交替：让偶数号的行先发后收，奇数号的行先收后发，然后各自翻转角色。这样总有一半线程在收，握手能够完成。

[5]视频画面时间区间：00:15:00–00:15:15。若为裁剪图，时间区间仍对应原视频画面。

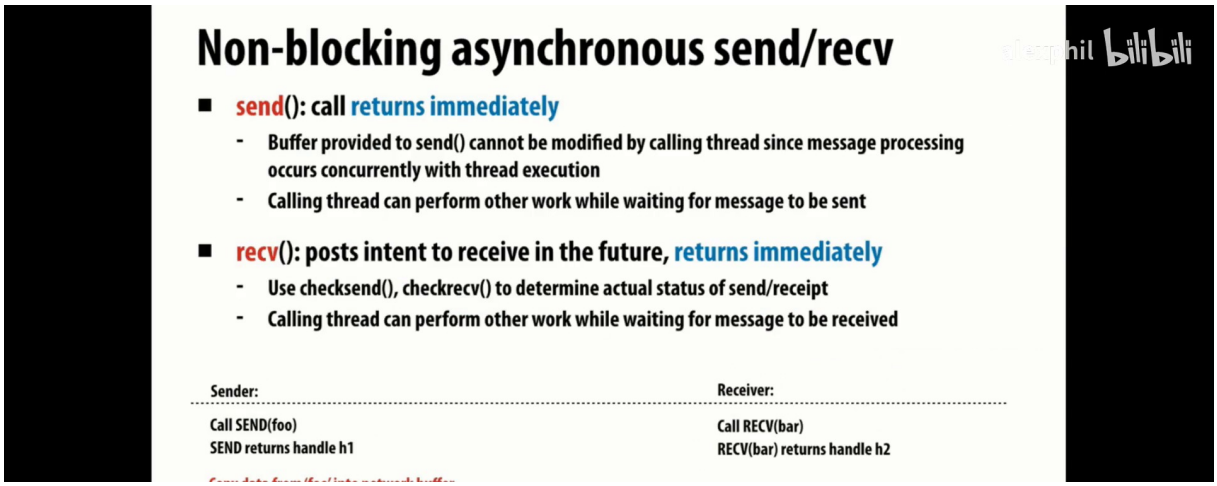


图 6: 非阻塞异步发送与接收：调用立即返回，稍后再确认完成[6]

阻塞发送的配对顺序必须由程序员保证

奇偶交替不是优化技巧，而是正确性要求。它把”每一对收发都有一方先就位”这件事编码进了线程编号。一旦线程拓扑变化（例如改成二维分块，每个块有上下左右四个邻居），奇偶规则就要重新设计。这也是消息传递被认为”僵硬”的具体体现。

2.3 非阻塞发送

另一种约定是非阻塞发送：调用立即返回，真正的传输在后台进行。

问题是这样可能堆积出无界的待发消息。因此通常返回一个令牌（句柄），发送方之后可以用它查询发送是否真的完成了。

还可以做得更彻底：收发两端都完全异步。接收方可以声明”我想收某个消息，好了告诉我”，然后去做别的事，等消息真正到达时触发一个回调。

讲者的评价是：如果你能设计出让人人有事可做的应用，这种机制很有用——不必干等消息到来。

非阻塞原语把等待从调用点移到了别处

阻塞版本的等待发生在 send/recv 调用处，代码结构因此被迫与通信顺序绑定。非阻塞版本把等待推迟到显式检查或同步点，让程序员能把通信与计算重叠。代价是需要管理句柄、处理未完成的请求，代码复杂度上升。

2.4 本章小结

阻塞发送要求收发双方会合，缓冲需求最低但容易死锁，奇偶交替是标准解法。非阻塞发送立即返回并给出句柄，可通过查询或回调确认完成，能把等待与计算重叠。

语义确定之后，就可以讨论一个更基础的问题：一次通信究竟要花多少时间？

[6] 视频画面时间区间：00:19:00–00:19:15。若为裁剪图，时间区间仍对应原视频画面。

3　延迟与带宽：两个必须分开的量

讲者用一个生活化的类比来建立直觉，这段讨论值得完整展开，因为它是后面所有通信分析的出发点。

3.1　公路类比

设想从旧金山开车到匹兹堡。车的速度是每小时 100 公里，路程 4000 公里，因此单程需要 40 小时。

但如果我们有多辆车呢？如果车之间保持 1 公里间距，那么任一时该公路上可以有 4000 辆车同时在路上，而到达率是每小时 100 人。

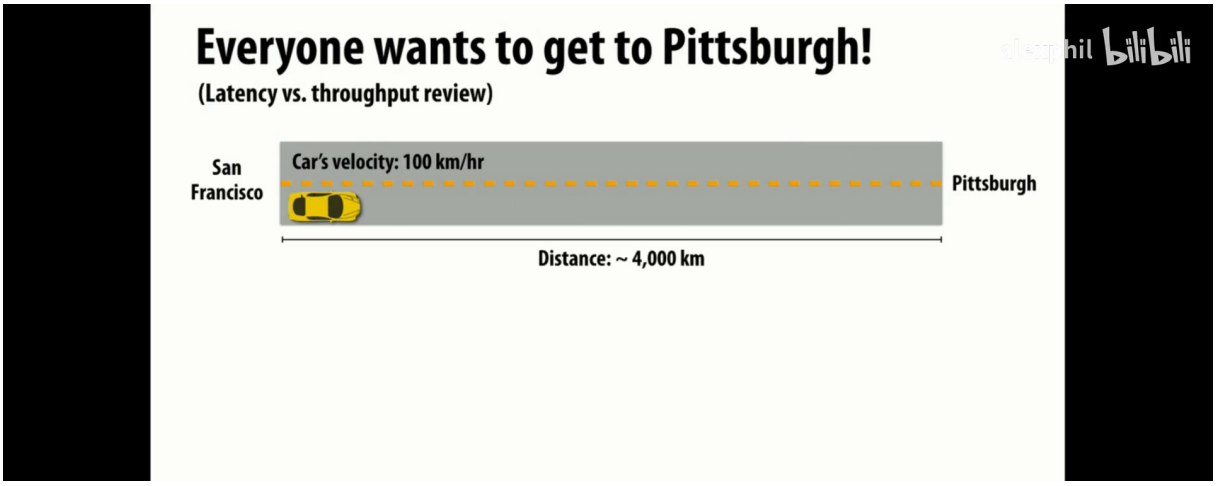


图 7: 延迟与吞吐：公路类比（时速 100 公里，全程 4000 公里）[7]



图 8: 公路类比：车速与车距共同决定吞吐[8]

这是提高吞吐而不改善延迟的经典例子。讲者的判断是：在大量系统设计里，改善吞吐比改善延迟容易得多。

[7] 视频画面时间区间：00:23:00–00:23:15。若为裁剪图，时间区间仍对应原视频画面。

[8] 视频画面时间区间：00:24:30–00:24:45。若为裁剪图，时间区间仍对应原视频画面。

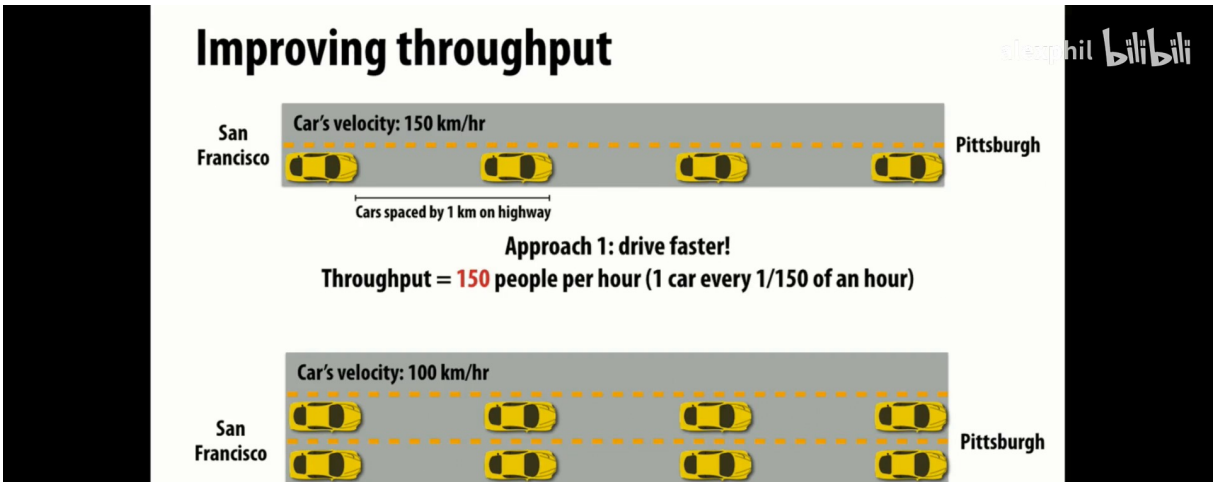


图 9: 提高车速到 150 公里/小时：同时改善延迟与吞吐[9]

当然也可以开得更快。若车速提升到 150 公里/小时，吞吐随之升到每小时 150 人，延迟也有所下降。或者可以修更多的路、把路修宽，同样能提高容量。这两种手段性质不同：开快车同时改善两者，加车只改善吞吐。

3.2 两个量的定义

讲者随即给出一组可用于任何系统的定义。

延迟是一次操作从开始到完成的端到端时间。它在系统各处都会出现：缓存缺失的处理时间、从 A 点发消息到 B 点的时间、从 Piazza 拿到回复的时间。

带宽是单位时间能完成的工作量，通常按字节而不是按操作计。

在纯串行情形下，带宽就是延迟的倒数（配上适当的单位）。但除此之外还有别的提高带宽的手段。

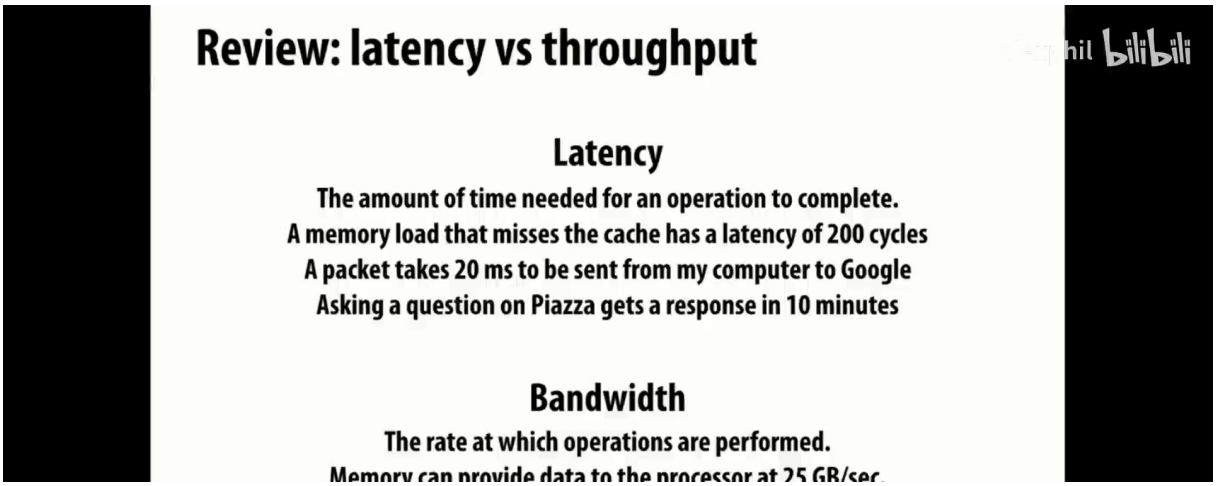


图 10: 延迟与吞吐的定义：前者是单次耗时，后者是单位时间完成量[10]

[9] 视频画面时间区间：00:25:00–00:25:15。若为裁剪图，时间区间仍对应原视频画面。

[10] 视频画面时间区间：00:26:00–00:26:15。若为裁剪图，时间区间仍对应原视频画面。

先判断瓶颈是延迟还是带宽

两者对应完全不同的改进手段。若瓶颈是延迟（关键路径上的一次访问），增加并行度无济于事，只能缩短路径或预取。若瓶颈是带宽（大量数据搬运），提高单次速度作用有限，应当增加并发通道或减少搬运总量。判断错方向，投入会完全浪费。

3.3 洗衣与流水线

讲者用一个更贴近并行计算的例子：洗衣服。

假设洗衣机 45 分钟、烘干机 60 分钟、折叠 15 分钟，那么完成一批衣服的总延迟是两小时。

要提升吞吐，最直接的办法是把资源加倍——两倍洗衣机、两倍烘干机，还得找个愿意帮你叠衣服的朋友。这样吞吐翻倍，而延迟一点没变。

更好的办法是流水线。第一批洗完后立刻开始第二批，不必等第一批全部完成。当然会有等待：烘干机要 60 分钟而洗衣机只要 45 分钟，所以第二批洗完后必须等 15 分钟才有空闲烘干机。

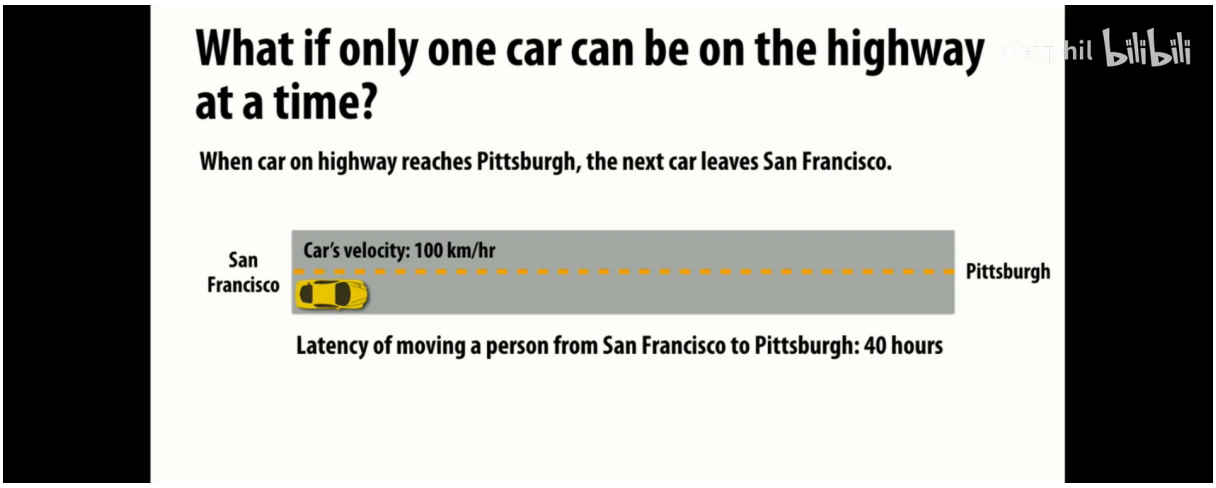


图 11：若公路上同时只能有一辆车会怎样：串行化的极端情形[11]



图 12：用洗衣流程提升吞吐：目标是最多大化多批衣服的吞吐[12]

[11] 视频画面时间区间：00:27:00–00:27:15。若为裁剪图，时间区间仍对应原视频画面。

[12] 视频画面时间区间：00:28:30–00:28:45。若为裁剪图，时间区间仍对应原视频画面。

关键结论是：

$$\text{Throughput}_{\max} = \frac{1}{\max_i L_i}$$

- L_i：第 i 个流水阶段的延迟，本例中洗衣机 45 分钟、烘干机 60 分钟、折叠 15 分钟；
- $\max_i L_i$：最慢阶段的延迟，即 60 分钟；
- 结果：这个系统最多每小时完成一批。

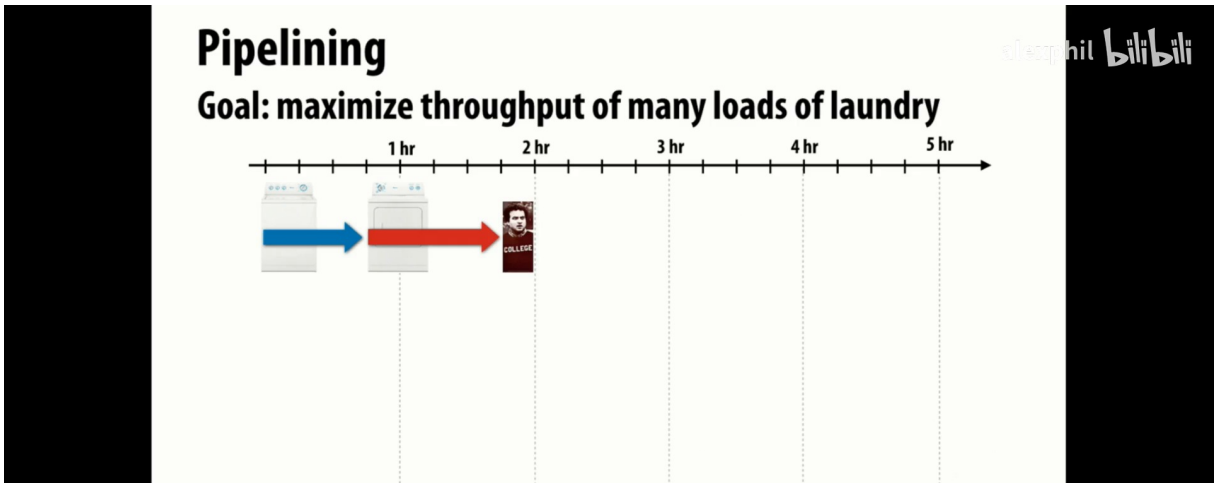


图 13: 流水线：让各阶段始终有活在干，吞吐由最慢阶段决定[13]

讲者指出好消息：吞吐翻倍而没有增加任何资源。坏消息是你每小时得花 15 分钟叠衣服——但洗衣机与烘干机反正闲着，不用白不用。

3.4 处理器也是这么做的

讲者说这正是处理器设计的原理：把指令执行拆成若干阶段——取指、译码、执行、写回。最基础的流水线只有这几级，而现代处理器有 15 到 20 级，每级耗时可能不同，并配有大量动态调度的执行单元。

用公路类比再讲一遍：可以把 4000 公里切成 4000 个一公里的区段，规定每段同时只能有一辆车。这其实很像铁路调度——把轨道分成离散区段，保证同一段上不会有两列车。

如果能不增加开销地把流水级做得更细，例如改成 500 米一段，那么同时可用的资源更多，吞吐还能翻倍。

[13] 视频画面时间区间：00:29:00–00:29:15。若为裁剪图，时间区间仍对应原视频画面。

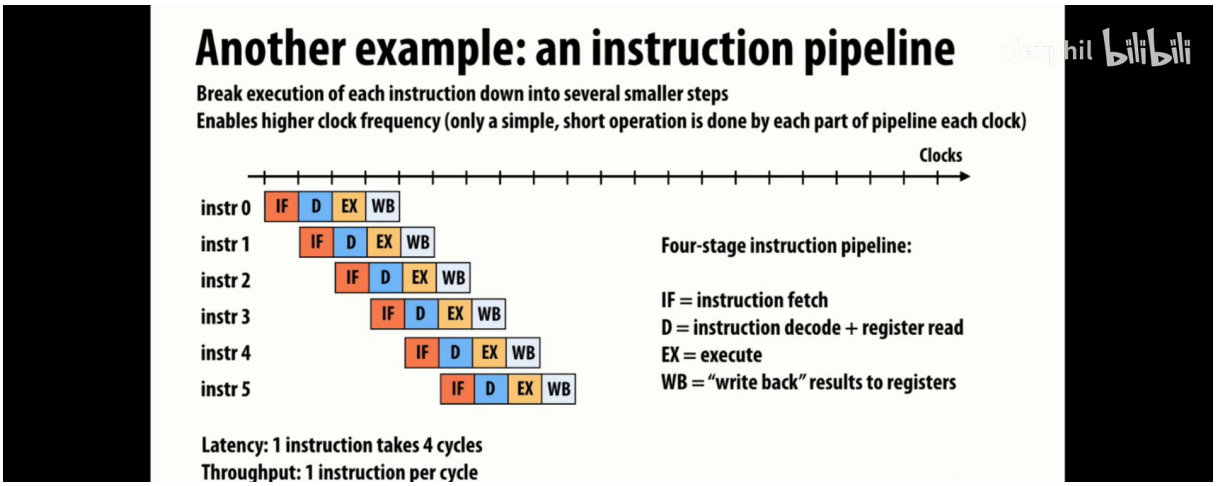


图 14: 指令流水线：把执行拆成取指、译码、执行、写回等阶段[14]

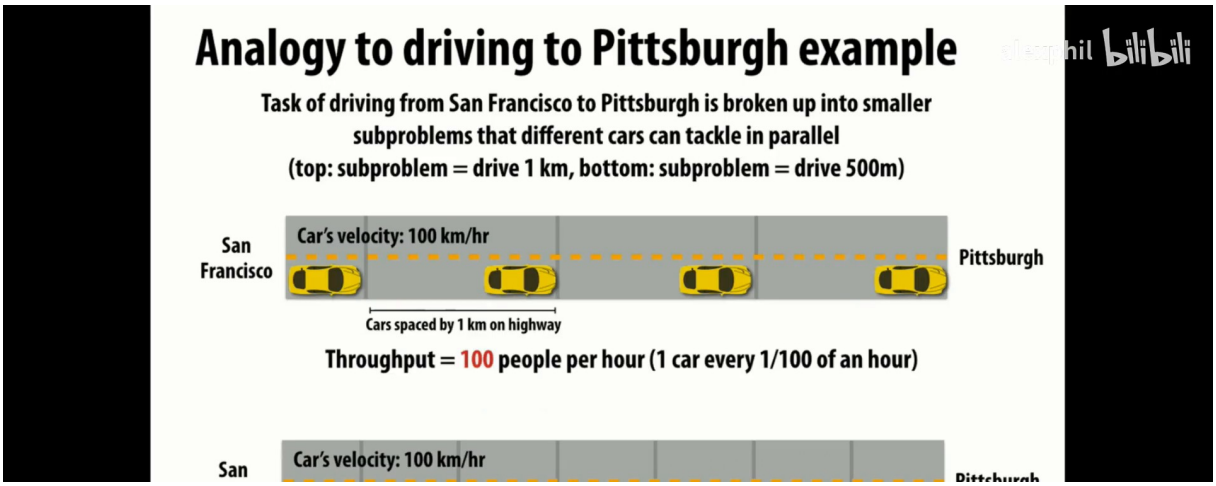


图 15: 与驾车类比对应：把公路切成区段当作流水阶段[15]

3.5 本章小结

延迟是单次操作的端到端时间，带宽是单位时间完成量，两者的改进手段不同。公路类比说明增加并发可以提升吞吐而不改善延迟；洗衣例子给出流水线的核心结论：吞吐上限由最慢阶段决定，而流水可以在不增加资源的情况下翻倍吞吐。

这个结论直接适用于通信。下一章把它写成可以用来估算的公式。

4 通信耗时的模型

4.1 固定开销加传输时间

讲者用公路类比建立通信的简单模型：发送一条 n 比特的消息。

[14] 视频画面时间区间：00:30:30–00:30:45。若为裁剪图，时间区间仍对应原视频画面。

[15] 视频画面时间区间：00:32:00–00:32:15。若为裁剪图，时间区间仍对应原视频画面。

$$T(n) = T_0 + \frac{n}{B}$$

- $T(n)$：发送 n 比特消息的总耗时；
- T_0：固定开销，包括消息从 A 到 B 的传播时间与协议处理，跨美国也要几秒的量级；
- n：消息的比特数；
- B：链路带宽，单位比特每秒。

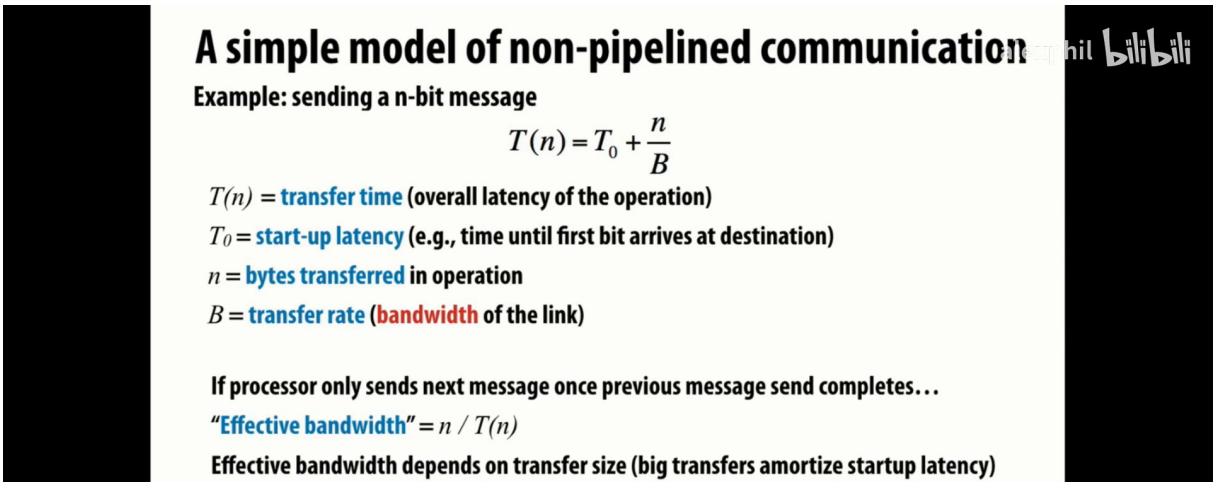


图 16: 非流水线通信的简单模型：$T(n) = t_0 + n/B$[16]

讲者说这是许多通信模型不错的第一近似。它立刻带来一条实用推论：如果把消息合并成数量更少、每条更大的消息，T_0 带来的影响在总量上就被摊薄了。

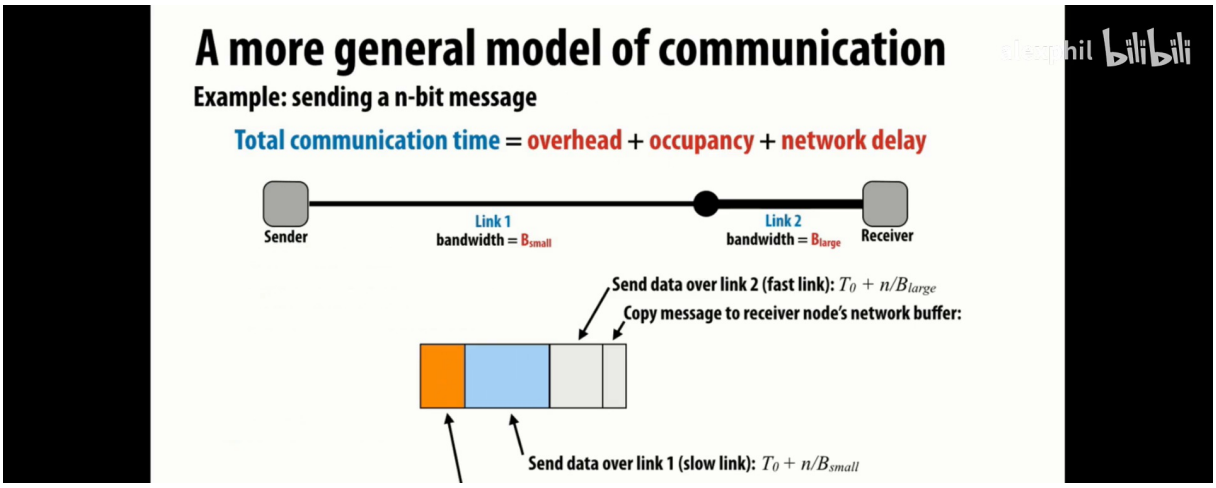


图 17: 更一般的通信模型：固定开销加传输时间，并考虑最慢链路[17]

他的原话是：几乎所有的通信介质都更擅长发送长消息而不是短消息。

[16] 视频画面时间区间：00:33:00–00:33:15。若为裁剪图，时间区间仍对应原视频画面。
[17] 视频画面时间区间：00:35:30–00:35:45。若为裁剪图，时间区间仍对应原视频画面。

4.2 有效带宽

由此可以定义有效带宽：用总字节数除以包含 T_0 的实际总时间。

$$B_{\text{eff}} = \frac{n}{T(n)} = \frac{n}{T_0 + n/B}$$

- B_{eff}：实际达到的带宽；
- n：本次传输的字节数；
- 分母包含固定开销，因此 B_{eff} 总是低于名义带宽 B；
- 当 n 很大时 B_{eff} 才趋近 B。

这就把”发小消息吃亏”变成可计算的量：消息越小，固定开销占比越高，有效带宽越低。

频繁的小消息是并行程序的隐形杀手

按上式，若每次只发几十字节，有效带宽可能只有名义带宽的百分之几。这在网格求解器里很容易发生：每个时间步都要交换边界，如果每步只交换一行而步数极多，T_0 就被重复支付了几千次。合并消息、减少同步次数，往往比优化计算本身收益更大。

4.3 最慢链路与流水线化通信

讲者进一步指出：通信系统里有快有慢，而吞吐通常由最慢的那条链路决定——就像洗衣例子里烘干机是关键瓶颈。

他画出一段通信的分解：橙色是建立调用（相当于 T_0）的开销，蓝色是最慢的那段链路，灰色是其余部分（下游链路与接收端固定成本）。从吞吐角度看，只有蓝色部分真正决定上限。

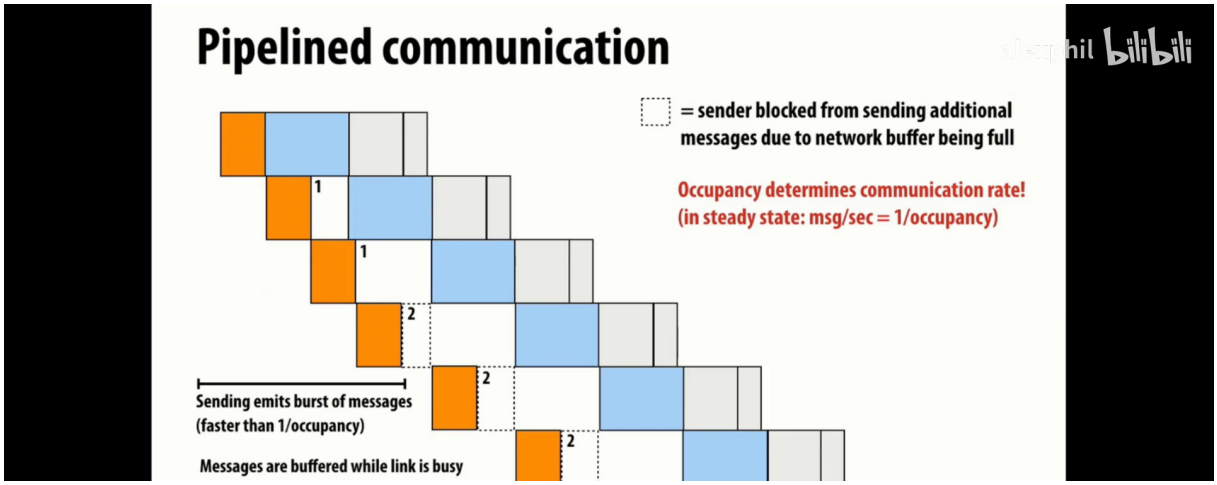


图 18：流水线化的通信：让最慢链路持续忙碌，其余开销被重叠[18]

好消息是可以在通信系统里也建流水线。之前之所以受限，是因为缺少足够多的离散部分来重叠，于是每次都重复支付开销。若能建立流水线，T_0 与下游操作都能与传输重叠，让最慢链路始终保持忙碌，有效带宽就趋近于那条链路的带宽。

[18] 视频画面时间区间：00:36:30–00:36:45。若为裁剪图，时间区间仍对应原视频画面。

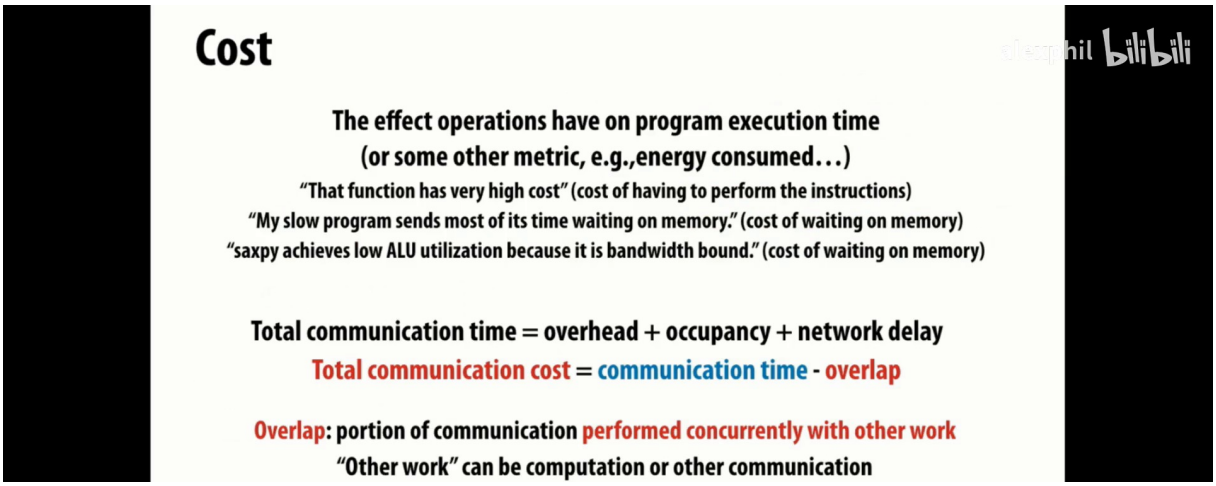


图 19: 成本的定义：各种操作对程序执行时间（或能耗）的影响[19]

流水线化通信的前提是消息可分割

把通信做成流水线要求消息能被切成多段，各段依次流过链路。这与”合并大消息”看似矛盾，实际解决的是不同层面的问题：合并处理 T_0 的摊薄，流水处理段间的重叠。两者可以并用——先合并成少数大消息，再让大消息内部流水传输。

4.4 本章小结

通信耗时可建模为固定开销加传输时间，有效带宽因此总低于名义带宽，小消息尤其吃亏。系统吞吐由最慢链路决定，而流水线化能让最慢链路持续忙碌、把其余开销重叠掉。这三条结论构成了后面所有优化的依据。

讲者在这里给出”成本”的总定义：成本是各种操作对程序执行时间的影响，也可以按能耗等其他资源来衡量。接下来要把并行系统放进内存层次的框架里看。

5 把并行系统看成扩展的内存层次

5.1 缓存既是资源也是成本

讲者提出一个统一的视角：通信在并行系统里扮演的角色，与缓存在单机里的角色同类。它提供了与其他线程交换数据的机制，这很有用；但同时也是一笔开销。

这解释了为什么我们总想最大化局部性——把正在操作的数据留在尽可能小、尽可能近、尽可能快的存储层次里。

[19] 视频画面时间区间：00:37:30–00:37:45。若为裁剪图，时间区间仍对应原视频画面。

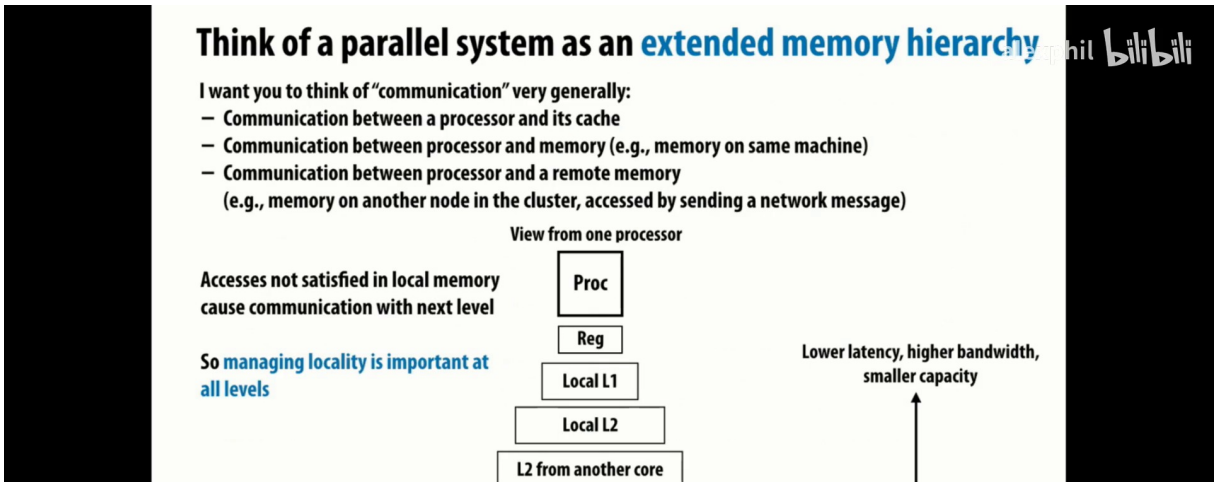


图 20: 把并行系统看作扩展的内存层次：越远的存储越慢但容量越大[20]

放大到大规模系统，除了本地内存，还有别的处理器所管理的内存——它们仍然可访问，只是更慢、容量更大。再往外，就是必须通过消息传递才能拿到的、存放在其他处理器上的信息。

无论哪一层，规律一致：越远越慢，越多越大。

5.2 固有通信与人为通信

讲者给出本讲最重要的分类。

固有通信是算法本身要求发生的通信。在网格求解器里，必须把相邻块的那一行数据传过去，否则算不出来。这部分无法消除。

人为通信是实现方式引入的、本可以避免的通信。它可能来自硬件设计、软件设计或系统设计的任何一部分，但在理想世界里本该不存在。

区分固有与人为通信决定优化方向

固有通信只能通过改变算法或分块方式来减少，例如换一种划分让边界更短。人为通信可以通过改实现来消除，例如提高传输粒度、调整数据布局、避免伪共享。把两者混为一谈，就会在错误的方向上投入——试图用调参解决算法层面的通信量。

[20] 视频画面时间区间：00:39:30–00:39:45。若为裁剪图，时间区间仍对应原视频画面。

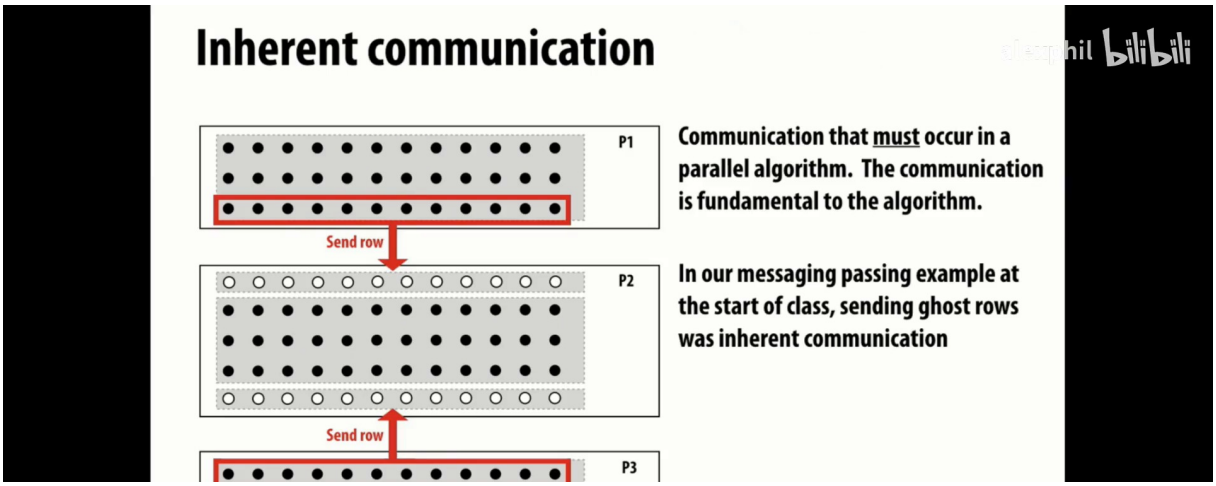


图 21: 固有通信：并行算法中必然发生的通信[21]

5.3 算术强度：把通信与计算放在一起看

讲者引入一个常用的比率：通信量与计算量之比。他习惯把它翻过来，称为算术强度：

$$\text{AI} = \frac{\text{计算量}}{\text{通信量}}$$

- 分子是完成的有用计算，例如浮点操作数；
- 分母是需要搬运的数据量，例如字节数；
- 比值越高，说明每一次数据搬运换来了更多计算，通信越不构成瓶颈。

他举了 SAXPY 作为反面例子：单精度 $a \times x + y$ 只做两次浮点操作（一次乘、一次加），却要读两个源向量、写一个目标向量。按任何口径算，都是每三次内存操作换两次算术操作。讲者的评语是：这不太好。

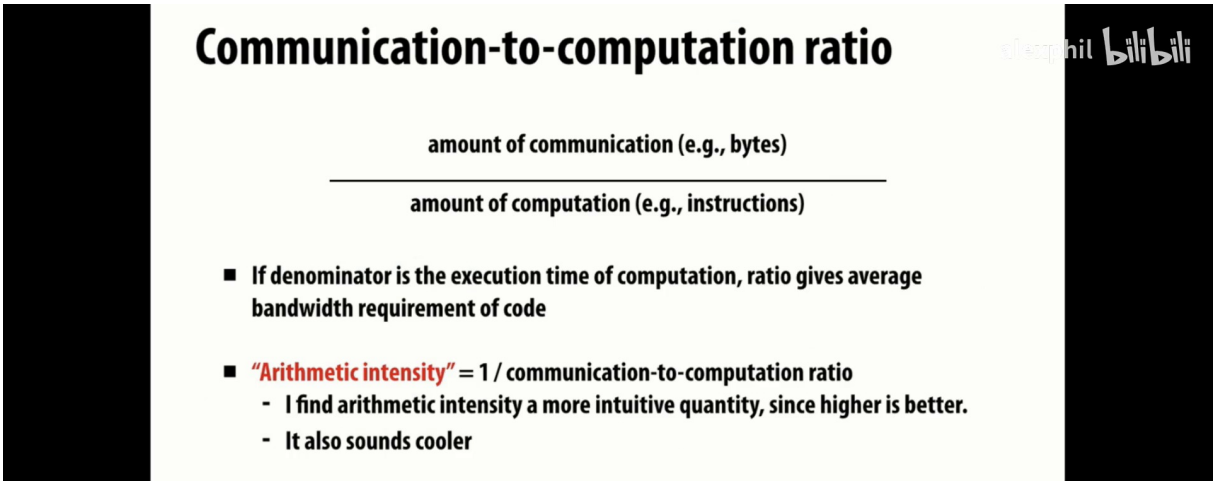


图 22: 通信与计算之比：算术强度的定义[22]

[21] 视频画面时间区间：00:42:30–00:42:45。若为裁剪图，时间区间仍对应原视频画面。
[22] 视频画面时间区间：00:43:00–00:43:15。若为裁剪图，时间区间仍对应原视频画面。

5.4 本章小结

并行系统可以视为扩展的内存层次：越远越慢、越大。在这个框架下，通信分为算法必然要求的固有通信与实现引入的人为通信，前者只能靠改变分块减少，后者可以靠改实现消除。算术强度把计算量与通信量放在同一个比值里，是判断通信是否构成瓶颈的通用指标。

有了这个指标，就可以比较不同分块方式的优劣。这正是下面两章的内容。

6 分块形状如何决定算术强度

6.1 行分块

讲者回到网格求解器，假设是 $N \times N$ 的网格，按行分给 P 个处理器。

每个处理器分到的计算量是 N^2/P。而它需要与相邻线程交换的数据是 $2N$——上下各一行。

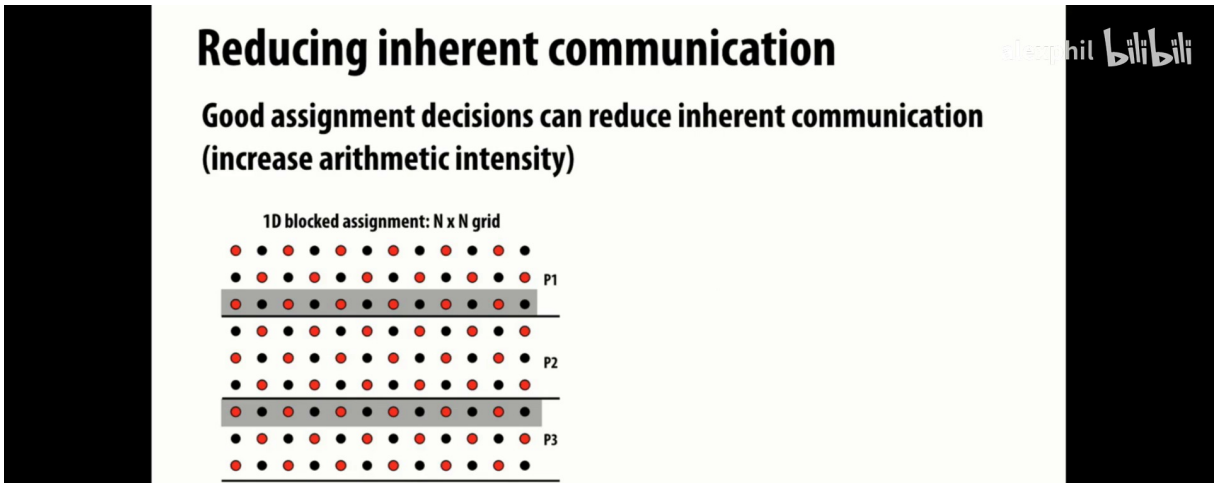


图 23: 减少固有通信：分配决策可以改变通信量[23]

于是算术强度为：

$$\mathrm{AI}_{\mathrm{row}} \propto \frac{N^2/P}{2N} = \frac{N}{2P}$$

- 分子 N^2/P：每个处理器负责的元素数；
- 分母 $2N$：交换两行所需的元素数；
- 结果与 N/P 成正比。

好消息是 N 越大，算术强度越高。直觉上也讲得通：N 大时每个处理器分到很多行，其中大部分是完全本地的，根本不需要通信。

坏消息是 P 越大，算术强度越低。讲者说这通常是普遍规律：越想并行，算术强度就越差。

6.2 交叉分配：一个反面教材

讲者接着分析另一种分配：让每个处理器负责第 1 行、第 4 行、第 5 行等等，交错分布。

每个处理器的计算量仍然是 N^2/P。但通信量呢？学生很快答出来：几乎是全部。

[23] 视频画面时间区间：00:44:30–00:44:45。若为裁剪图，时间区间仍对应原视频画面。

理由是：每算一个元素，都要把它的值向上和向下各传一次。因此算术强度只有约 0.5。

$$\mathrm{AI}_{\text{interleaved}} \approx \frac{1}{2}$$

这个数字与 N、P 都无关，恒定为一个极差的值。讲者的结论很直接：这是个很糟糕的主意。

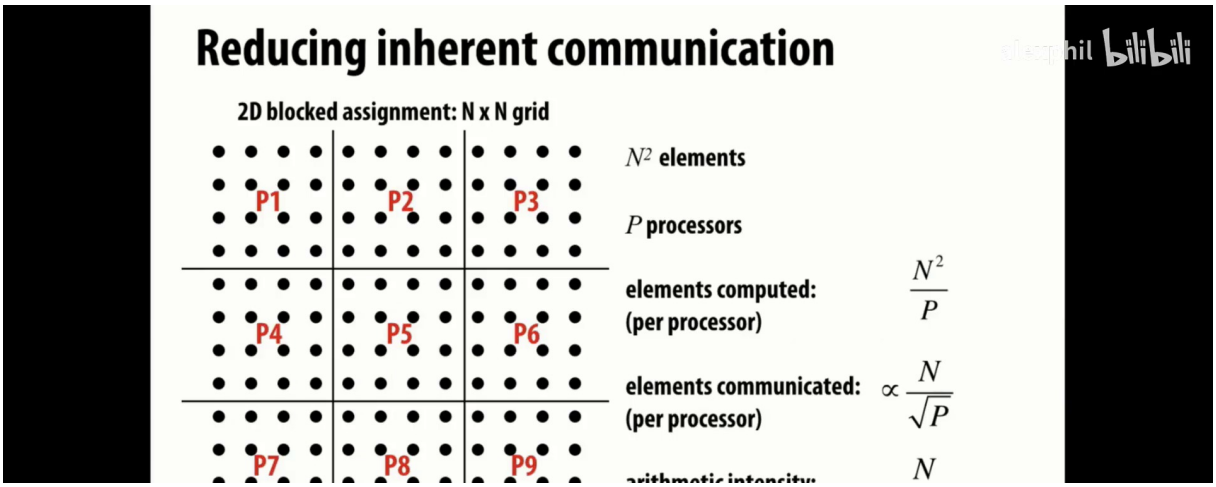


图 24：二维分块分配：用面积换周长，大幅减少边界通信[24]

6.3 方形分块：面积与周长之比

讲者问：能否做得比行分块更好？他提示想象水滴——表面张力会让水滴倾向于最小化表面积与体积之比。

答案是把网格分成方形的块，而不是长条。

这样每个处理器分到的计算量不变，但通信量正比于块的周长，而周长随面积的平方根增长。因此算术强度变成：

$$\mathrm{AI}_{\text{2D}} \propto \frac{N^2/P}{\sqrt{N^2/P}} = \frac{N}{\sqrt{P}}$$

- 通信量正比于周长，即 $\sqrt{N^2/P} = N/\sqrt{P}$；
- 结果与 $N/\sqrt{P}$ 成正比；
- 与行分块的 $N/(2P)$ 相比，多出了一个 $\sqrt{P}$ 的优势。

讲者的评价是这对扩展性很有利：随着处理器增加，性能受到的负面影响逐步变小。

通信正比于边界、计算正比于面积

这条原则可以推广到任意维度：如果计算量随体积增长而通信量只随表面积增长，那么块越大，算术强度越高。三维时通信正比于边界面（面积的平方根关系变成 $P^{2/3}$），四维更好。因此分块形状的选择本质上是在选择一个几何量——在固定体积下让表面积最小。

[24] 视频画面时间区间：00:48:30–00:48:45。若为裁剪图，时间区间仍对应原视频画面。

6.4 固有通信也可以减少

讲者在这里做了一个重要的澄清：虽然这部分通信是”固有的”，但并不意味着不能对它做优化。

改变分块形状就是减少固有通信的手段：通信量仍然由算法决定必须有，但不同划分下它的总量不同。所以”固有”指的是不可消除，而不是不可优化。

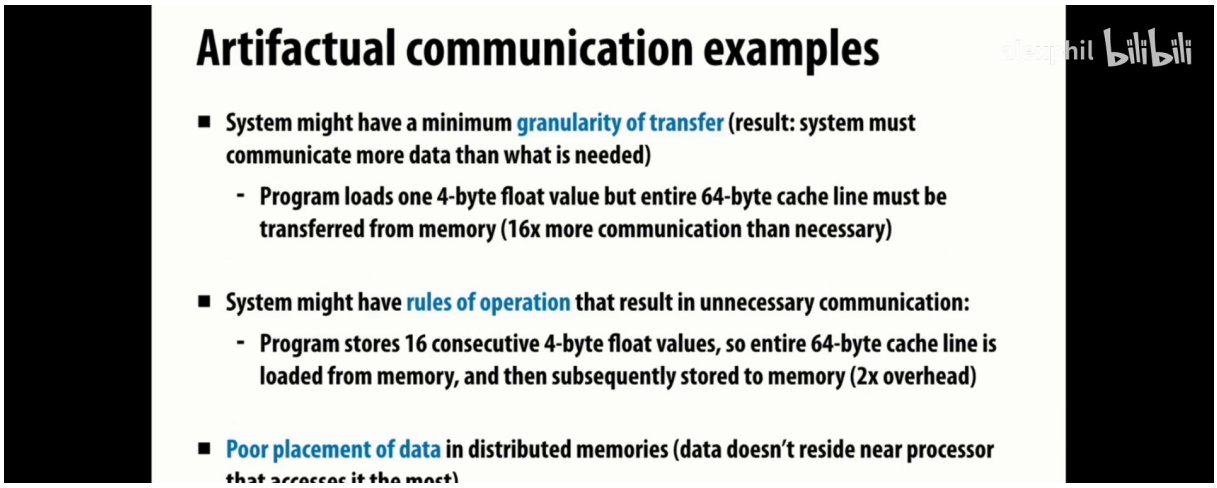


图 25：人为通信的例子：最小传输粒度等带来的额外搬运[25]

6.5 本章小结

按行分块时算术强度正比于 $N/(2P)$，交叉分配只有约 0.5，方形分块则是 $N/\sqrt{P}$。差别来自通信量正比于边界而计算量正比于面积。固有通信虽然不可消除，但可以通过改变分块形状来减少总量。

形状问题解决之后，还有一类通信与算法无关，纯由实现细节产生。那是下一章的内容。

7 人为通信：缓存行、缺失与遍历顺序

7.1 最小传输粒度

讲者先给出最普遍的一种人为通信：系统往往有最小传输粒度，例如一条缓存行。

即使只想读其中的一个字节，也要把整行搬过来。如果不巧只有这一个字节有用，这份额外的搬运就完全是浪费。

写的情况更糟。讲者举了 SAXPY 的例子：即使你直接指向目标地址去写，每次进入新的一行时系统仍要先把它读进来，因为它并不知道你即将写满整行。于是第一次写操作会把原有数据从内存搬到缓存，再更新、再写回。这些旧数据你根本用不上。

写分配导致的读放大

写命中的前提是数据已在缓存里，因此写缺失会先触发一次读填充。对连续写满整个缓存行的场景，这次读完全是多余的。避免办法是让写操作按完整缓存行对齐、成批进行，或用非临时写指令绕过缓存填充。这类浪费在计数器、标志位这类零散写入上尤其明显。

[25] 视频画面时间区间：00:50:30–00:50:45。若为裁剪图，时间区间仍对应原视频画面。

7.2 第四类缺失：通信缺失

讲者回顾了从 15-213 课程就该掌握的三类缺失。冷缺失是第一次访问某数据，缓存里没有，通常无法避免。容量缺失是工作集太大装不进缓存。冲突缺失是缓存的组织方式（例如组相联索引）导致它没能选到最好的淘汰对象，从纯程序角度看本不该发生。

现在加入第四类：通信缺失。指在系统更高层次上，我们付了本不需要的额外通信代价。

讲者提醒这些定义在细究时会互相重叠。

7.3 从工作集看通信

讲者用一张图把通信放在工作集框架里：系统有一部分能以很高带宽工作，当工作集超出它的容量就掉进更慢的资源，依此类推。最底下是本来就无法避免的固有通信与冷缺失。

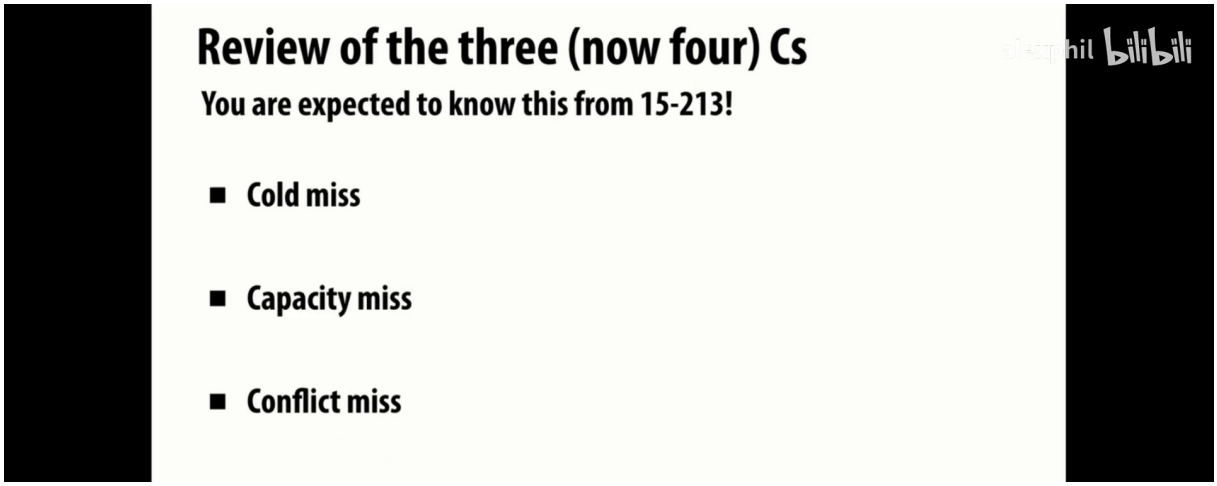


图 26: 回顾四类缺失：冷缺失、容量缺失、冲突缺失与通信缺失[26]

他指出这张图其实与之前的内存山是同一件事：取一个截面，横轴是工作集大小，小工作集留在 L1 里飞快，变大就掉进 L2、L3，最后卡在内存。

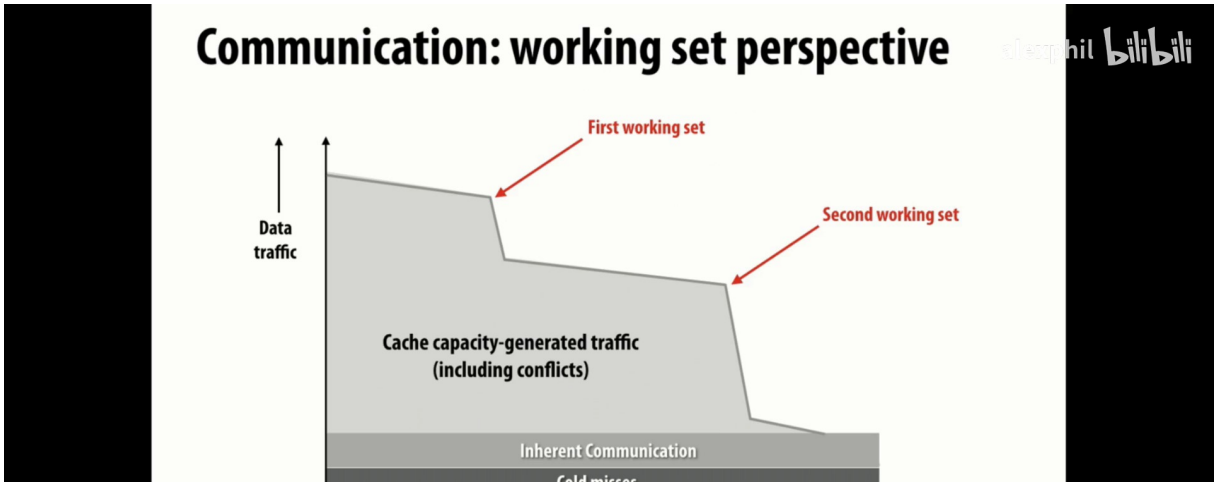


图 27: 从工作集角度看通信：跨越层次时成本骤升[27]

[26] 视频画面时间区间：00:52:30–00:52:45。若为裁剪图，时间区间仍对应原视频画面。

[27] 视频画面时间区间：00:54:30–00:54:45。若为裁剪图，时间区间仍对应原视频画面。

7.4 行主序遍历的容量缺失

讲者用一个具体例子把上面说得抽象的结论落地。假设按行主序访问网格：访问 [0] → [1] → [2] → [3]，然后回到下一行。

若缓存能装 24 个元素，且按每 4 个元素一组对齐，那么遍历完一整行再回到下一行开头时，第一行开头的数据已经被挤出去了——它在完成上一行时被淘汰。

于是每次跨行都恰好缺失一次。讲者的评价是：空间局部性还算不错，一次缺失三次命中；短期时间局部性也可以。但正是因为容量缺失，跨行时无法享受复用。

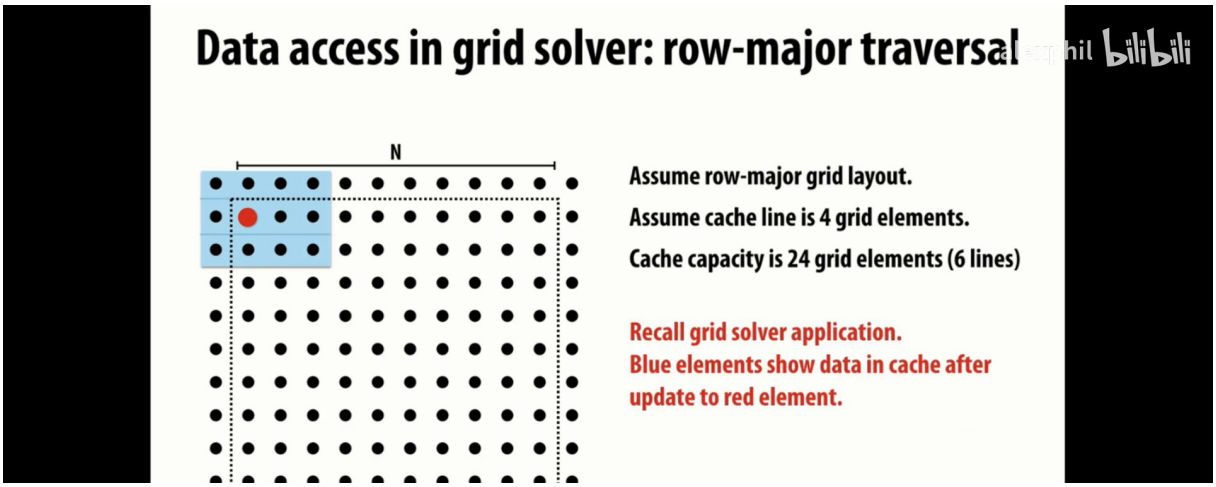


图 28: 网格求解器的行主序访问：每次跨行都要重新缺失[28]

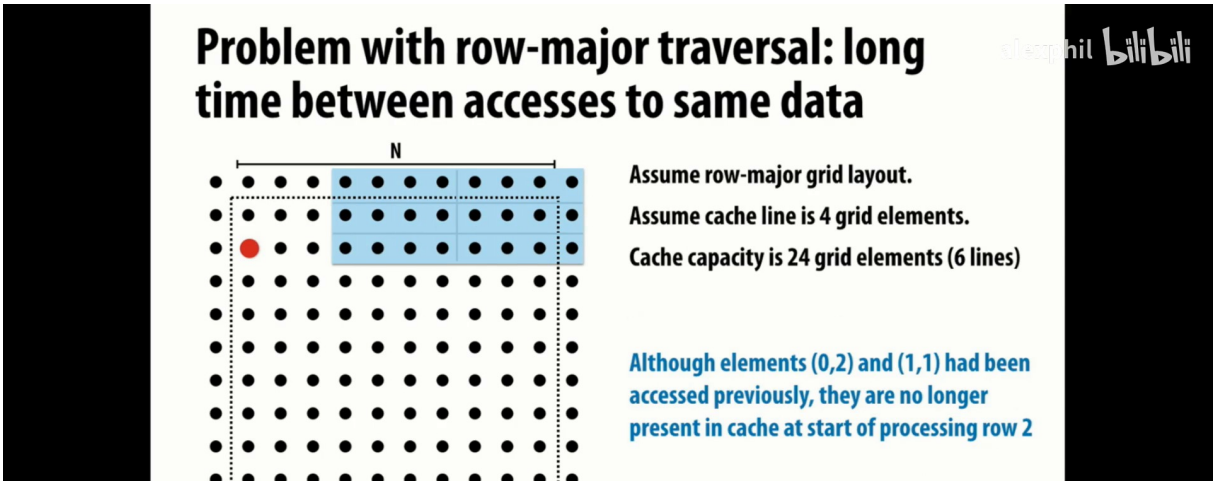


图 29: 行主序的问题：同一数据两次使用之间间隔太长[29]

7.5 改遍历顺序与循环融合

解法是改变访问顺序：先处理前几列，让这些列的上下行复用发生在短时间内，再回来处理剩下的列。行主序或列主序都可以，关键在于不要按标准顺序傻走。

[28] 视频画面时间区间：00:57:00–00:57:15。若为裁剪图，时间区间仍对应原视频画面。

[29] 视频画面时间区间：00:58:00–00:58:15。若为裁剪图，时间区间仍对应原视频画面。

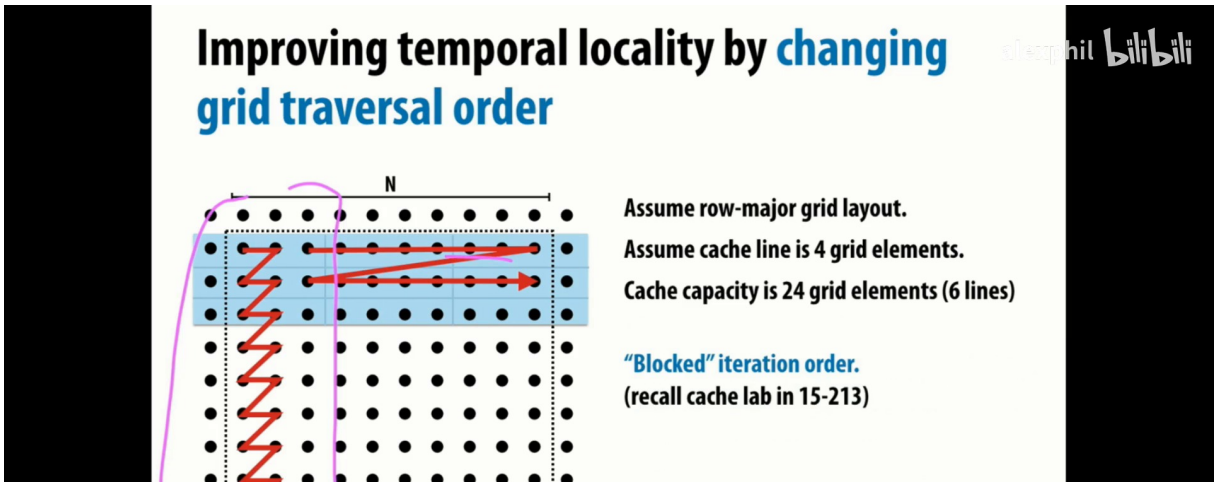


图 30：改变网格遍历顺序以改善时间局部性[30]

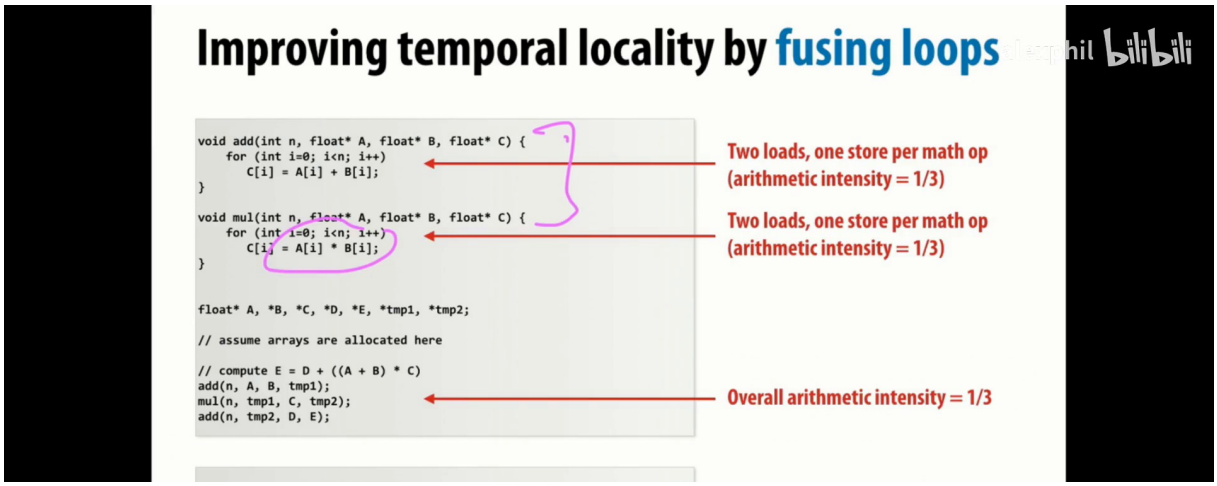


图 31：循环融合改善时间局部性：把两个遍历合成一个[31]

另一个手段是共享数据以提高算术强度：把操作同一批数据的任务安排在一起，让这份数据只被搬一次。

[30] 视频画面时间区间：00:59:30–00:59:45。若为裁剪图，时间区间仍对应原视频画面。

[31] 视频画面时间区间：01:01:00–01:01:15。若为裁剪图，时间区间仍对应原视频画面。

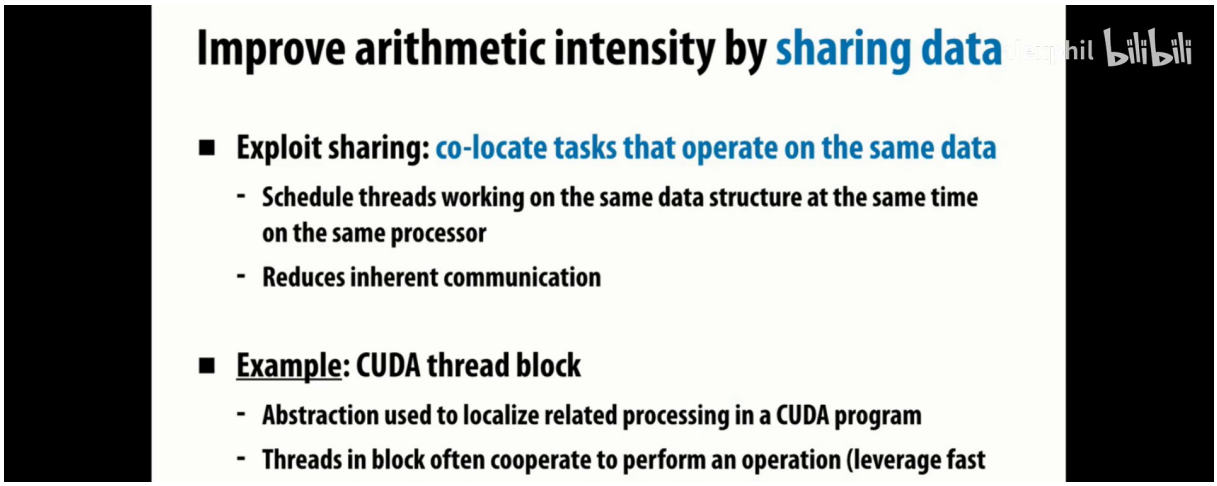


图 32: 共享数据以提高算术强度[32]

7.6 数据并行写法与性能的张力

讲者在这里指出了一个现实矛盾。用高度抽象的方式写代码很舒适：想问题时就想着数组级别的操作。比如 Python 的 numpy 里直接写 `A + B`，三维数组也知道你要逐元素相加。CUDA 的线程启动也可以这样想：先做一次向量加，再做一次向量加，读起来很优雅。

问题是这种写法从性能角度看很糟。

讲者的解释是：这类抽象把”数据在哪、怎么搬”完全隐藏了，而通信与局部性恰恰是决定性能的全部。因此数据并行的优雅与性能之间常常需要取舍。

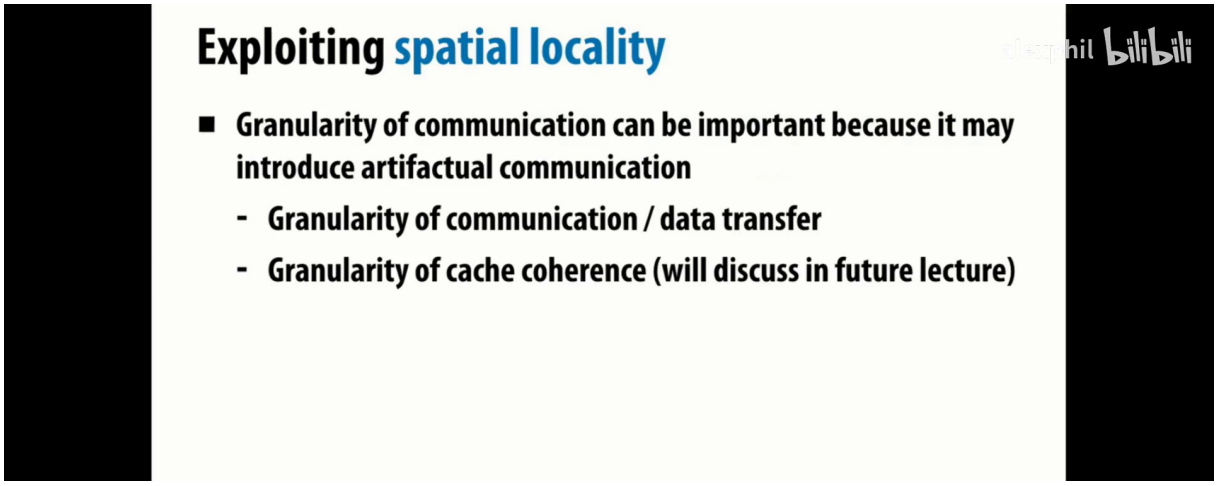


图 33: 利用空间局部性：通信粒度也很重要[33]

7.7 本章小结

人为通信来自最小传输粒度（读一个字节要搬整行、写之前先读入旧行）与布局不佳。第四类缺失——通信缺失——把这类浪费纳入缺失分类。行主序遍历会导致跨行的容量缺失，改变遍历顺序或融合循环可以修复。而抽象层次越高，越容易在无意中制造这类浪费。

[32] 视频画面时间区间：01:03:30–01:03:45。若为裁剪图，时间区间仍对应原视频画面。

[33] 视频画面时间区间：01:05:30–01:05:45。若为裁剪图，时间区间仍对应原视频画面。

8 竞争与伪共享

8.1 伪共享的机制

讲者在最后讨论了另一类人为成本：竞争，也就是多个线程同时访问同一个共享资源。

其中最容易忽视的一种叫伪共享：不同线程操作的数据虽然不同，却恰好落在同一条缓存行里。缓存行是失效与传输的单位，因此一个线程写自己的数据，会让另一个线程持有的同一行副本失效，迫使它重新取数。两个线程之间其实没有任何数据依赖，却在缓存协议层面互相干扰。

伪共享的判据是缓存行而不是变量

判断是否存在伪共享，要看多个线程的写目标是否落在同一缓存行，而不是看它们是否访问同一个变量。两个完全无关的计数器若相邻存放，同样会互相拖慢。解决办法是填充（padding）或按缓存行对齐分配，把每个线程的写目标推到独立的行上。

8.2 原子操作与热点

讲者提到另一类问题：用原子操作做累加之类的更新时，所有线程都会打向同一个地址，形成热点。硬件必须把这些操作串行化，代价随线程数上升。

给出的对策是避免原子操作，改用更高层次的并行原语：先让各线程算局部结果，最后再合并。这与前面反复出现的”部分和加归约”是同一个模式。

全局计数器是并行程序的典型瓶颈

每个线程都去自增一个全局计数器，等于把所有线程串行化在同一个地址上。随着线程数增加，这份争用会成为无可避免的上限。可行替代是每线程一个局部计数器、最后汇总，或使用分片计数器（sharded counter）把争用分散到多个地址。

8.3 本章小结

竞争与伪共享属于人为成本中最隐蔽的一类：它们不会让结果出错，只让性能莫名下降，且难以从代码表面看出来。判据是缓存行与共享地址，而非变量与逻辑依赖。避免手段是分散写目标、用局部累加替代原子操作。

讲者最后的时间分配说明这一讲已覆盖了成本的主要来源与对策。

9 总结与延伸

讲者收尾时说：我们走过了成本的各个部分，以及如何减少它们、提高性能。

把全讲串起来，是一条从”通信不可避免”到”如何把它压到最低”的完整链条。通信成本足以抹掉并行收益，而它在共享内存、消息传递、GPU 三种形态中本质相同，只是距离不同。消息传递把这一点暴露得最清楚：按行分块后每个线程必须复制邻居边界，收敛判断还需要一次全局归约，而阻塞发送若不配奇偶交替就会死锁。延迟与带宽必须分开看待——公路类比说明增加并发可以只提升吞吐；洗衣例子给出流水线的核心结论：吞吐上限由最慢阶段决定。通信耗时可建模为 $T_0 + n/B$，因此合并消息能摊薄固定开销，有效带宽总低于名义带宽。并行系统可视为扩展的内存层次，通信分为算法必然要求的固有通信与实现引入的人为通信；算术强度把两者放在同一个比值里。分块形状决定算术强度：行分块是 $N/(2P)$，交叉分配只有 0.5，方形分块是 $N/\sqrt{P}$，差别来自通信正比边

界而计算正比面积。最后，最小传输粒度、遍历顺序、伪共享与原子操作构成了人为成本的主要来源。

本讲义在讲者材料之上做的压缩，有四点需要与讲者原话区分：

- 通信优化的抓手是几何量而非代码细节。分块形状决定边界长度，边界长度决定通信量。这比微调传输参数有效得多，也解释了为什么行分块与方形分块的差距会随处理器数扩大。
- 固定开销 T_0 的代价随调用次数放大，而不是随数据量放大。因此减少同步次数、合并消息，往往比增大带宽更直接。这与 Amdahl 定律的结构相同：不能并行的那部分反复出现，就是上限。
- 固有通信可以优化但不可消除。改变划分能让它变少，但算法要求的跨块数据依赖始终存在。设计时应先承认这部分不可避免，再看剩余空间。
- 抽象层次越高，越容易制造人为通信。numpy 式数组操作与 CUDA 的高层线程启动都很优雅，但它们隐藏了数据布局，而布局恰恰决定通信量。

这堂课的边界在于它讨论的是成本的结构，而不是某个具体系统的调优手册。讲者明确表示伪共享、原子操作这些细节会在后续课程与作业中继续展开。

两个可以直接从本讲数据提出的下一步问题：其一，方形分块需要处理器数能凑成完全平方，实际划分时如何处理不能整除的情形，代价有多大？其二，既然算术强度随 P 增大而下降，那么强扩展（固定问题规模加机器）本质上必然失效，弱扩展（问题规模随机器增长）是否才是唯一可行的路径？这两个问题都指向接下来的缓存一致性与扩展性讨论。